

CO4_AT1_TECHNICAL PRESENTATION

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

Bellman Principle of Optimality

Policy Evaluation and Policy Improvement

Value Iteration vs. Policy Iteration

Industrial applications

1. Introduction

Dynamic Programming (DP) is a mathematical approach used in Reinforcement Learning to solve **sequential decision-making problems**. It helps an agent find the best actions by calculating the expected future rewards for different states and actions.

DP is mainly used when the **environment model is known**, including transition probabilities and rewards.

The two important DP algorithms are:

- **Policy Iteration**
- **Value Iteration**

2. Bellman Principle of Optimality

The **Bellman Principle of Optimality** states:

An optimal policy has the property that, regardless of the initial state and first decision, the remaining decisions must form an optimal policy.

In simple words, **an optimal decision now should lead to optimal decisions in the future.**

Bellman Optimality Formula

$$V^*(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s')]$$

Where:

- $V^*(s)$ = Optimal value of state s
- $R(s,a)$ = Immediate reward
- γ = Discount factor
- $P(s'|s,a)$ = Probability of reaching next state s'
- $V^*(s')$ = Value of next state
- a = Action

3. Policy Evaluation

Policy Evaluation calculates how good a given policy is.

For every state, it calculates the expected return when the agent follows the current policy.

Formula

$$V\pi(s) = R(s, \pi(s)) + \gamma \sum P(s'|s, \pi(s)) V\pi(s')$$

Simple Steps

- Start with an initial policy.
- Initialize the value of each state.
- Calculate new state values.
- Repeat until the values become stable.

4. Policy Improvement

Policy Improvement tries to find a better action for every state using the values obtained during policy evaluation.

Formula

$$\pi'(s) = \operatorname{argmax}[R(s,a) + \gamma \sum P(s'|s,a) V\pi(s')]$$

If the new policy is better than the old policy, replace the old policy.

Process

Initial Policy → Policy Evaluation → Policy Improvement → Better Policy

This process continues until the policy becomes optimal.

5. Policy Iteration

Policy Iteration combines **Policy Evaluation** and **Policy Improvement**.

Algorithm

- Initialize a random policy.
- Evaluate the policy.
- Improve the policy.
- Check whether the policy has changed.
- If it changed, repeat.
- If it does not change, the policy is optimal.

Flow

Initial Policy



Policy Evaluation



Policy Improvement



Policy Changed?



Yes

No



Repeat

Optimal Policy

6. Value Iteration

Value Iteration directly updates the value of each state using the Bellman Optimality Equation.

Formula

$$V_{k+1}(s) = \max_a [R(s,a) + \gamma \sum_{s'} P(s'|s,a) V_k(s')]$$

The process continues until the values converge.

After convergence, the optimal policy is selected.

Simple Steps

- Initialize state values.
- Calculate the value for every possible action.
- Select the maximum value.
- Update the state value.
- Repeat until convergence.
- Extract the optimal policy.

7. Value Iteration vs Policy Iteration

Feature	Value Iteration	Policy Iteration
Main idea	Updates state values	Evaluates and improves policy
Policy	Extracted after convergence	Improved during iterations
Steps	Value update	Evaluation + Improvement
Complexity	Generally simpler	More computation per iteration
Result	Optimal values and policy	Optimal policy

Easy Memory Trick

Value Iteration:

Values $\rightarrow$ Optimal Policy

Policy Iteration:

Policy → Evaluation → Improvement → Optimal Policy

8. Industrial Applications

Dynamic Programming is used in many real-world applications.

Robotics

Used for:

- Robot path planning
- Navigation
- Movement optimization

Autonomous Vehicles

Used for:

- Route planning
- Decision-making
- Collision avoidance

Manufacturing

Used for:

- Machine scheduling
- Production planning
- Resource allocation

Logistics

Used for:

- Delivery route optimization
- Warehouse management

- Vehicle scheduling

⚡ Energy Management

Used for:

- Smart grid optimization
- Battery management
- Energy consumption reduction

9. Simple Python Code

This is a **short and easy Value Iteration example**.

Simple Value Iteration Example

```
states = ["A", "B", "Goal"]
```

```
actions = {
```

```
    "A": ["right"],
```

```
    "B": ["right"],
```

```
    "Goal": []
```

```
}
```

```
reward = {
```

```
    ("A", "right"): 0,
```

```
    ("B", "right"): 10
```

```
}
```

```
next_state = {
```

```
    ("A", "right"): "B",
```

```
    ("B", "right"): "Goal"
```

```
}
```

```
gamma = 0.9
```

```
V = {"A": 0, "B": 0, "Goal": 0}
```

```

for i in range(10):
    new_V = V.copy()

    for s in ["A", "B"]:
        a = actions[s][0]
        ns = next_state[(s, a)]
        new_V[s] = reward[(s, a)] + gamma * V[ns]

V = new_V

print("State Values:")
for s in states:
    print(s, "=", V[s])

```

Expected Output

State Values:

A = 8.1

B = 9.0

Goal = 0

The values show that **B is closer to the goal and has a higher value**, while A receives its value through the future reward from B.

10. Overall Conclusion

Dynamic Programming provides a systematic method for solving sequential decision-making problems in Reinforcement Learning. The **Bellman Principle of Optimality** forms the mathematical foundation of DP. **Policy Evaluation** calculates the value of a policy, while **Policy Improvement** makes the policy better. **Policy Iteration** repeatedly performs these two steps, whereas **Value Iteration** directly updates state values using the Bellman Optimality Equation. Dynamic Programming is widely applicable in robotics, autonomous vehicles, manufacturing, logistics, and energy management.

3. Deep Q-Networks for Intelligent Decision-Making

- **DQN Architecture**
- **Experience Replay**
- **Target Networks**

1. Introduction

A **Deep Q-Network (DQN)** is a Reinforcement Learning algorithm that combines **Q-Learning** with a **Deep Neural Network**.

DQN is useful when the number of states is very large or when states are represented by complex data such as images.

Instead of storing a Q-table, DQN uses a neural network to estimate the **Q-value** of each possible action.

The basic idea is:

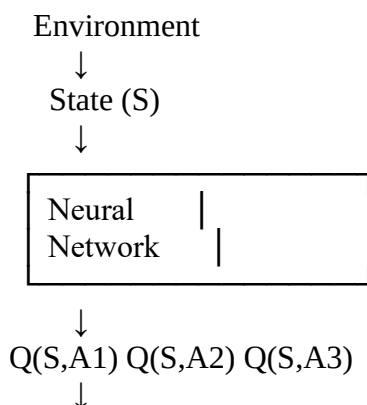
State → **Neural Network** → **Q-values** → **Best Action**

2. DQN Architecture

A DQN consists mainly of:

1. **Input Layer**
 - Receives the current state of the environment.
2. **Hidden Layers**
 - Neural network processes the state.
 - Learns important features.
3. **Output Layer**
 - Produces Q-values for all possible actions.

Simple Architecture



Best Action
↓
Environment

For example, if a robot has three actions:

- Move Forward
- Move Left
- Move Right

The network may produce:

Move Forward = 5.2
Move Left = 2.8
Move Right = 7.1

The agent chooses **Move Right** because it has the highest Q-value.

3. Q-Value

The Q-value represents the expected future reward for taking an action in a particular state.

Q-Learning Formula

$$Q(s,a)=Q(s,a)+\alpha[r+\gamma a'\max Q(s',a')-Q(s,a)]$$

Where:

- s = Current state
- a = Current action
- r = Reward
- s' = Next state
- α = Learning rate
- γ = Discount factor

In DQN, the neural network learns to approximate these Q-values.

4. DQN Loss / Target Formula

DQN uses a target value to train the neural network.

$$y=r+\gamma a'\max Q_{\text{target}}(s',a')$$

The loss is:

$$L(\theta)=[y-Q_{\text{online}}(s,a)]^2$$

Where:

- y = Target Q-value

- Q_{online} = Q-value predicted by the current network
- Q_{target} = Q-value from the target network

5. Experience Replay

Experience Replay is a technique used to make DQN training more stable and efficient.

The agent stores its experiences in a **Replay Memory**.

Each experience contains:

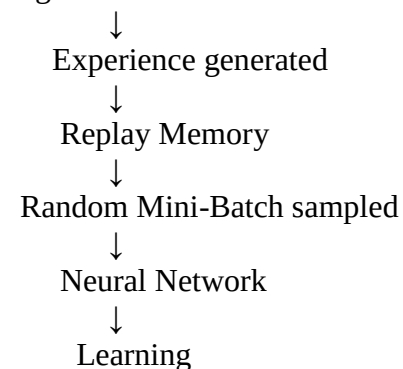
$(s, a, r, s', done)$

Where:

- s = Current state
- a = Action
- r = Reward
- s' = Next state
- $done$ = Whether the episode has ended

Working

Agent interacts with environment



Why Experience Replay?

It:

- Reduces correlation between consecutive experiences.
- Improves training stability.
- Allows useful experiences to be reused.
- Improves sample efficiency.

6. Target Network

DQN uses **two neural networks**:

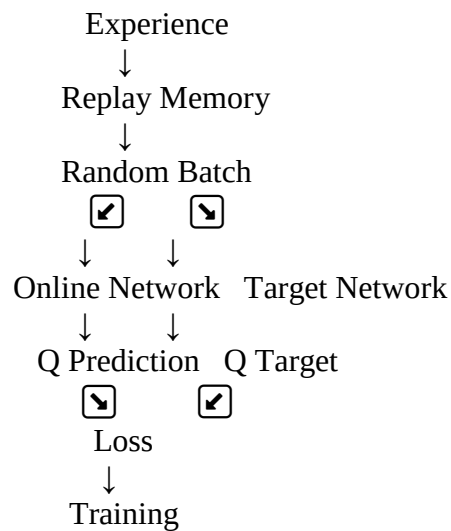
1. Online Network

- Updated frequently.
- Learns from the experiences.

2. Target Network

- Used to calculate stable target Q-values.
- Updated periodically using the online network.

Architecture



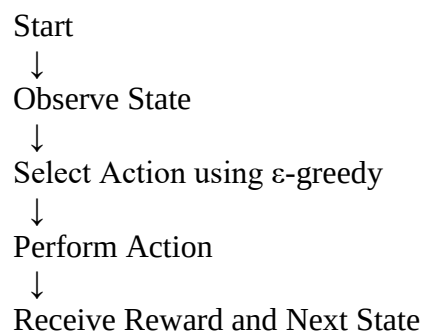
Why Target Network?

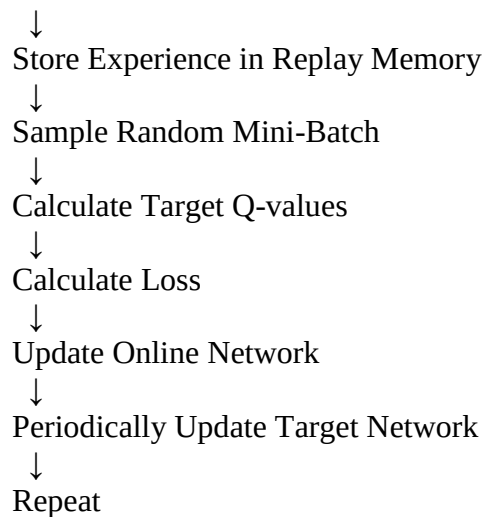
If the same network is used to calculate both the prediction and target, the target can change continuously.

The target network provides a **more stable learning target** and helps prevent unstable training.

7. DQN Working Process

The complete DQN process is:





8. Epsilon-Greedy Strategy

DQN commonly uses **ϵ -greedy** action selection.

With probability ϵ :

Explore → Select a random action.

With probability $1-\epsilon$:

Exploit → Select the action with the highest Q-value.

Formula

$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \text{argmax}_a Q(s,a) & \text{otherwise} \end{cases}$

This allows the agent to discover new actions while also using actions that it already knows are good.

9. Simple Python Code

The following is a **short conceptual DQN code segment** using a simple neural network.

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Create DQN
model = Sequential([
    Dense(24, activation="relu", input_shape=(4,)),
    Dense(24, activation="relu"),
    Dense(2, activation="linear")
])

model.compile(optimizer="adam", loss="mse")
```

```
# Example state
state = np.array([[0.1, 0.2, 0.3, 0.4]])

# Predict Q-values
q_values = model.predict(state, verbose=0)

print("Q-values:", q_values)
print("Selected Action:", np.argmax(q_values))
```

Sample Output

```
Q-values: [[0.25 0.48]]
Selected Action: 1
```

Here, the network produces Q-values for **2 possible actions**.

Since:

$0.48 > 0.25$

the agent selects **Action 1**.

Note: This short code demonstrates the DQN neural-network component. A complete DQN implementation would additionally include an environment, replay memory, target network, training loop, and ϵ -greedy exploration.

10. Industrial Applications

Robotics

DQN can help robots learn:

- Navigation
- Obstacle avoidance
- Path planning
- Pick-and-place decisions
- Autonomous movement

Example

A warehouse robot receives:

State: Current position + obstacle information

Actions: Forward, Left, Right

Reward: Positive for reaching the destination and negative for collisions.

The DQN learns which action is best for different situations.

Autonomous Systems

DQN can be used for:

- Autonomous driving decisions
- Lane-changing decisions
- Route selection
- Collision avoidance
- Traffic management

For example, an autonomous vehicle can use sensor information as the state and choose actions such as:

Accelerate → Brake → Maintain Speed → Change Lane

11. Advantages of DQN

- Handles large state spaces.
- Uses neural networks for function approximation.
- Can learn directly from complex states.
- Experience replay improves learning stability.
- Target networks improve training stability.
- Useful in robotics and autonomous systems.

12. Limitations of DQN

- Requires significant training data.
- Neural network training can be computationally expensive.
- Sensitive to hyperparameters.
- Basic DQN is mainly suitable for **discrete action spaces**.
- Training can sometimes be unstable.

13. Conclusion

A **Deep Q-Network** combines Q-Learning with deep neural networks to make intelligent decisions in complex environments. **Experience Replay** allows the agent to learn from stored experiences, while the **Target Network** provides stable Q-value targets. DQN is widely useful in robotics, autonomous vehicles, navigation, and other intelligent decision-making systems.